

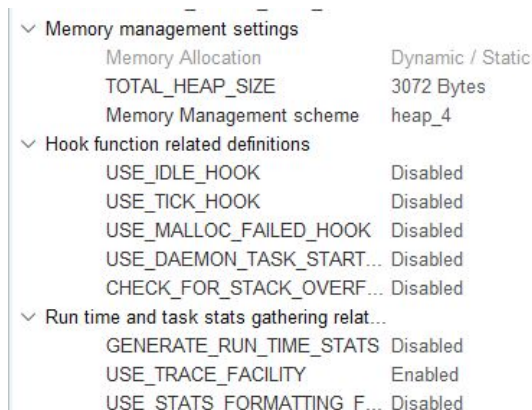
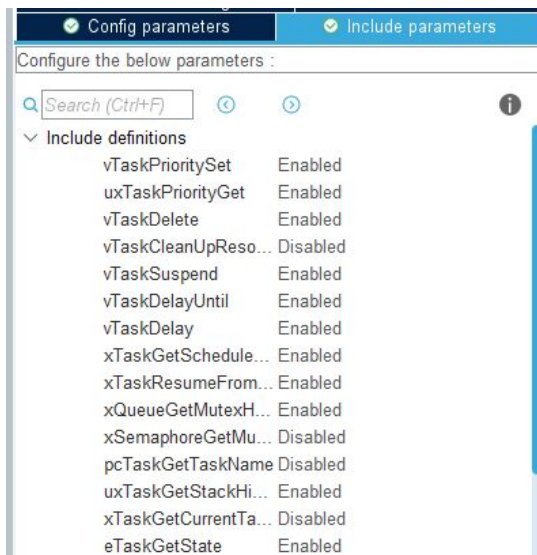
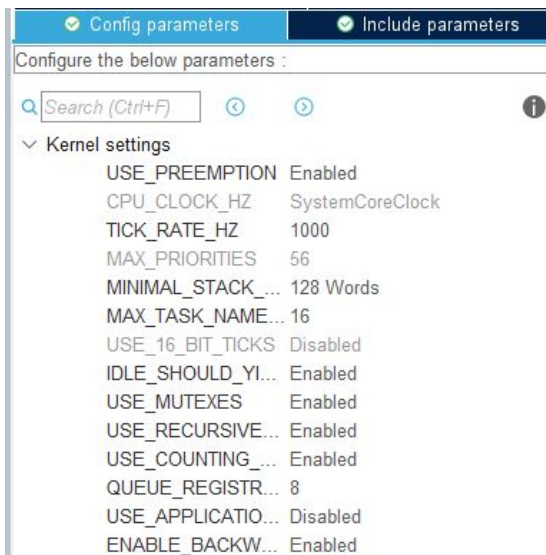
El sistema operativo y las tareas/threads

Tareas - Interrupciones

Usamos FreeRTOS con capa CMSIS OS V2 desde Cube IDE



FREERTOS Mode and Configuration



Menú para el agregado de Tareas

✓ User Constants	✓ Tasks and Queues	✓ Timers and Semaphores
✓ Config parameters	✓ Include parameters	✓ Advanced settings

Tasks

Task N...	Priority	Stack ...	Entry F...	Code G...	Param...	Allocati...	Buffer ...	Control...
default...	osPrior...	128	StartD...	Default	NULL	Dynamic	NULL	NULL

Default task, mainly in charge of middlewares init (when some are selected).
Can be modified but not removed!
Double-click to edit and modify.

Add

Delete

```

275 /* USER CODE BEGIN Header_comenzarTareaParpadeo */
276 /**
277  * @brief Function implementing the tareaParpadeo thread.
278  * @param argument: Not used
279  * @retval None
280  */
281 /* USER CODE END Header_comenzarTareaParpadeo */
282 void comenzarTareaParpadeo(void *argument) {
283     /* USER CODE BEGIN comenzarTareaParpadeo */
284     /* Infinite loop */
285     for (;;) {
286         osDelay(500);
287         HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
288     }
289     /* USER CODE END comenzarTareaParpadeo */
290 }
291

```

✓ User Constants	✓ Tasks and Queues	✓ Timers and Semaphores
✓ Config parameters	✓ Include parameters	✓ Advanced settings

Tasks

Task N...	Priority	Stack ...	Entry F...	Code G...	Param...	Allocati...	Buffer ...	Control...
default...	osPrior...	128	StartD...	Default	NULL	Dynamic	NULL	NULL
tareaParpadeo	osPriorityLow	128	comenzarTareaParpadeo	Default	NULL	Dynamic	NULL	NULL

Task Name

tareaParpadeo

Priority

osPriorityLow

Stack Size (Words)

128

Entry Function

comenzarTareaParpadeo

Code Generation Option

Default

Parameter

NULL

Allocation

Dynamic

Buffer Name

NULL

Control Block Name

NULL

Add

Delete

OK

Cancel

Las tareas/threads se generan a partir de funciones

- Recordemos que las tareas son funciones de C con un bucle infinito (nunca retornan)
- Cada tarea tiene un TCB y un stack propio asignado en el Heap que reservó el sistema operativo
- En ese stack puede guardar sus variables locales y, si llama a funciones, allí hará el stacking

Todo lo que declaremos aquí va a durar el tiempo de ejecución de la tarea y va a ser local a esa tarea.
Podríamos declarar más de una tarea aprovechando la misma función (sólo compartirán variables static)

Verán, como cualquier función, las variables globales y podrán compartir información con otras Tareas a través de ellas

Aquí implementaremos el código que la tarea debe realizar

Y nunca deberíamos llegar a este punto, sin BORRAR la tarea

```
void TareaA(void* args) {  
  
    for (;;) {  
  
    }  
  
}
```

Estados de las tareas

- Mejoremos el diagrama de los estados de las tareas que vimos anteriormente
- En un sistema con un sólo núcleo una sólo tarea por vez puede estar corriendo
- Esa será la tarea con el estado **“RUNNING”**
- Cuando no está corriendo, una tarea puede estar en alguno de los siguientes estados:
 - **READY** Está lista (y quiere) para que el scheduler la ponga a correr.
 - **BLOCKED** Si por algún motivo debe esperar un evento y el scheduler no debe tomarla en cuenta para hasta que ese evento ocurra.
 - **SUSPENDED** (o Inactive en CMSIS OS) Suspendida, hasta que no se la reanude no volverá a necesitar ejecutarse (i.e. sale de las tareas analizadas por el scheduler).

```
typedef enum {  
    osThreadInactive      = 0, ///  
    osThreadReady         = 1, ///  
    osThreadRunning       = 2, ///  
    osThreadBlocked       = 3, ///  
    osThreadTerminated    = 4, ///  
    osThreadError         = -1, ///  
} osThreadState_t;
```

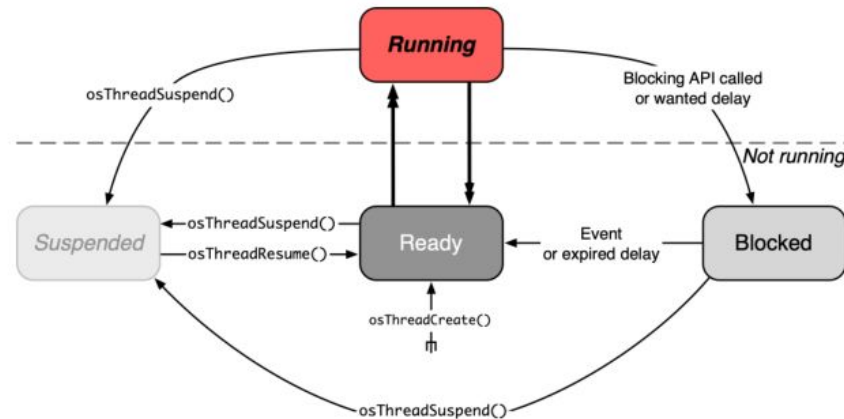


Figure 23.7: The possible states of a thread in FreeRTOS

Estrategia de Scheduling

- El RTOS configurado con CubeMX (o el FreeRTOS instalado “como viene”) tienen por default la estrategia **Preemptive con prioridades y Time Slice**
- Cada Tarea tiene una **prioridad fija**, asignada al crearla. El OS no puede cambiar esa prioridad (pero puede modificarse usando `osThreadSetPriority()`)
- El scheduler **cambiará la tarea cuando una de mayor prioridad pase al estado “READY”**, aunque la que se esté ejecutando no pase voluntariamente al estado Blocked o Suspended
- Entre tareas de igual prioridad se usará el time slice. Cuando se pasa la “tajada” el scheduler va a seleccionar la próxima tarea de una lista de READYS y le va a asignar nuevamente una tajada. Si no hay ninguna, pone a correr la tarea IDLE.
- Por defecto, la time slice y el tick del RTOS son iguales y de 1ms
- Cuando una tarea quiere ceder la ejecución puede usar la función `osTaskYield()` o generar un delay.

<code>configUSE_PREEMPTION</code>	<code>configUSE_TIME_SLICING</code>	Scheduling algorithm
1	1 or undefined	<i>Prioritized preemptive scheduling with time slicing</i>
1	0	<i>Prioritized preemptive scheduling without time slicing</i>
0	any value	<i>Cooperative scheduling</i>

Creación de tareas

```
/// Attributes structure for thread.
typedef struct {
    const char            *name;    ///< name of the thread
    void                  *cb_mem;   ///< memory for control block
    uint32_t               cb_size;  ///< size of provided memory for control block
    void                  *stack_mem; ///< memory for stack
    uint32_t               stack_size; ///< size of stack
    osPriority_t            priority;  ///< initial thread priority (default: osPriorityNormal)
    TZ_ModuleId_t           tz_module; ///< TrustZone module identifier
    uint32_t               reserved;  ///< reserved (must be 0)
} osThreadAttr_t;
```

```
osThreadId_t osThreadNew (osThreadFunc_t func, void *argument, const osThreadAttr_t *attr) {
    prio = (UBaseType_t)osPriorityNormal;
    name = attr->name;
    prio = (UBaseType_t)attr->priority;
    stack = attr->stack_size / sizeof(StackType_t);
    xTaskCreate ((TaskFunction_t)func, name, (uint16_t)stack, argument, prio, &hTask)
}
```

“Resumen” de la
función osThreadNew

Argumentos de las tareas

- El argumento es puntero a void para que se pueda castear cualquier cosa en ese formato y pasárselo a la tarea.
- Un ejemplo: dos tareas para hacer parpadear **distintos leds a distinta frecuencia** a partir de **la misma función**

```
/* USER CODE BEGIN PV */  
struct configParpadeo_t {  
    GPIO_TypeDef *puerto;  
    uint32_t pin;  
    uint32_t periodo;  
};  
struct configParpadeo_t conf1 =  
    {GPIOA, GPIO_PIN_5, 500};  
struct configParpadeo_t conf2 =  
    {GPIOA, GPIO_PIN_6, 1500};  
/* USER CODE END PV */
```

Task	
Task Name	tareaParpadeo
Priority	osPriorityLow
Stack Size (Words)	128
Entry Function	entryTareaParpadeo
Code Generation Option	Default
Parameter	&conf1
Allocation	Dynamic
Buffer Name	NULL
Control Block Name	NULL
OK Cancel	

Task	
Task Name	tareaParpadeo2
Priority	osPriorityLow
Stack Size (Words)	128
Entry Function	entryTareaParpadeo
Code Generation Option	Default
Parameter	&conf2
Allocation	Dynamic
Buffer Name	NULL
Control Block Name	NULL
OK Cancel	

Argumentos de las tareas

- El argumento es puntero a void para que se pueda castear cualquier cosa en ese formato y pasárselo a la tarea.
- Un ejemplo: dos tareas para hacer parpadear **distintos leds a distinta frecuencia** a partir de **la misma función**

```
/* creation of tareaParpadeo */
tareaParpadeoHandle = osThreadNew(
    entryTareaParpadeo,
    (void*) &conf1,
    &tareaParpadeo_attributes    );

/* creation of tareaParpadeo2 */
tareaParpadeo2Handle = osThreadNew(
    entryTareaParpadeo,
    (void*) &conf2,
    &tareaParpadeo2_attributes    );
```

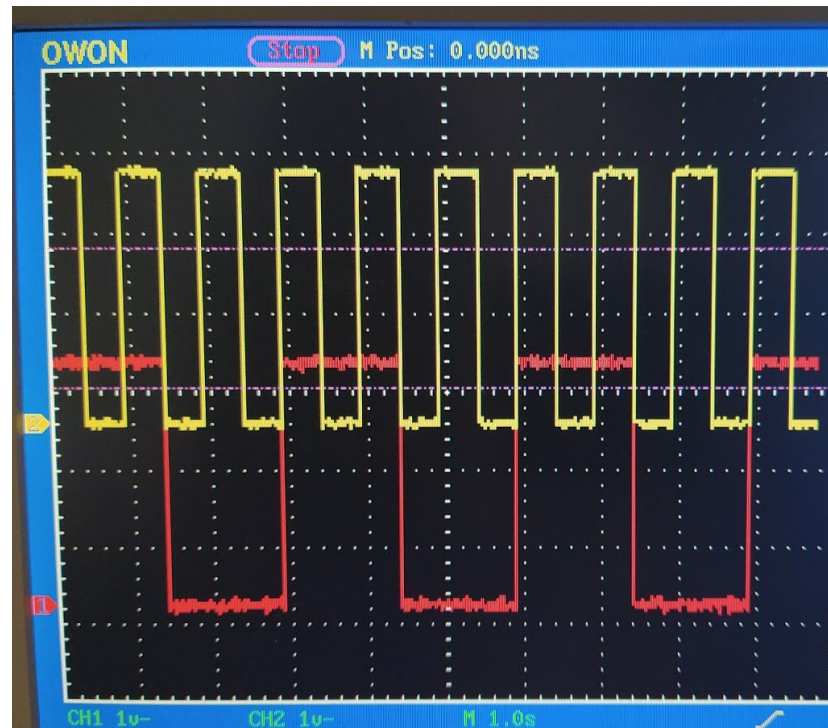
```
void entryTareaParpadeo(void *argument)
{
    /* USER CODE BEGIN entryTareaParpadeo */
    struct configParpadeo_t conf =
        *((struct configParpadeo_t*) argument);
    /* Infinite loop */
    for(;;)
    {
        HAL_GPIO_TogglePin(conf.puerto, conf.pin);
        osDelay(pdMS_TO_TICKS(conf.periodo));
    }
    /* USER CODE END entryTareaParpadeo */
}
```

Argumentos de las tareas

```
/* Variables globales */
struct configParpadeo_t conf1 =
{GPIOA, GPIO_PIN_5, 500};
struct configParpadeo_t conf2 =
{GPIOA, GPIO_PIN_6, 1500};

/* Instanciación de tareas en el main()*/
osThreadNew(entryTareaParpadeo, (void*) &conf1, ...);
osThreadNew(entryTareaParpadeo, (void*) &conf2, ...);

/* Función de las tareas */
void entryTareaParpadeo(void *argument){
    struct configParpadeo_t conf = *argument;
    for(;;){
        HAL_GPIO_TogglePin(conf.puerto, conf.pin);
        osDelay(pdMS_TO_TICKS(conf.periodo));
    }
}
```



Segmento: ¡Hágalo usted mismo!



- Instalar dos tareas
- Hacer titilar un LED con cada una a distinta frecuencia

Análisis del Stack

Explorando el stack

- Exploremos el stack de las tareas del ejemplo anterior
- Le dimos una “profundidad” (depth) de 128 palabras o $128 \times 4 = 512$ bytes
- ¿Está bien esto? Para saberlo deberíamos averiguar
 - El espacio que ocupan todas sus variables locales
 - Todas las funciones que llama (y aquellas que son llamadas por ellas) (call-graph)
 - El stack que necesitan todas las funciones en ese call-graph
 - Y por último calcular el peor caso
- Y ahora, ¿Quién podrá ayudarnos?

Explorando el stack

- Afortunadamente, el compilador de ARM tiene toda esta información y hace las cuentas para resolverlo por nosotros

Static Stack Analyzer × Console Problems Executables Debugger Console

laboratorio_tasks_programa_2.elf - /laboratorio_tasks_programa_2/Debug - Oct 30, 2022

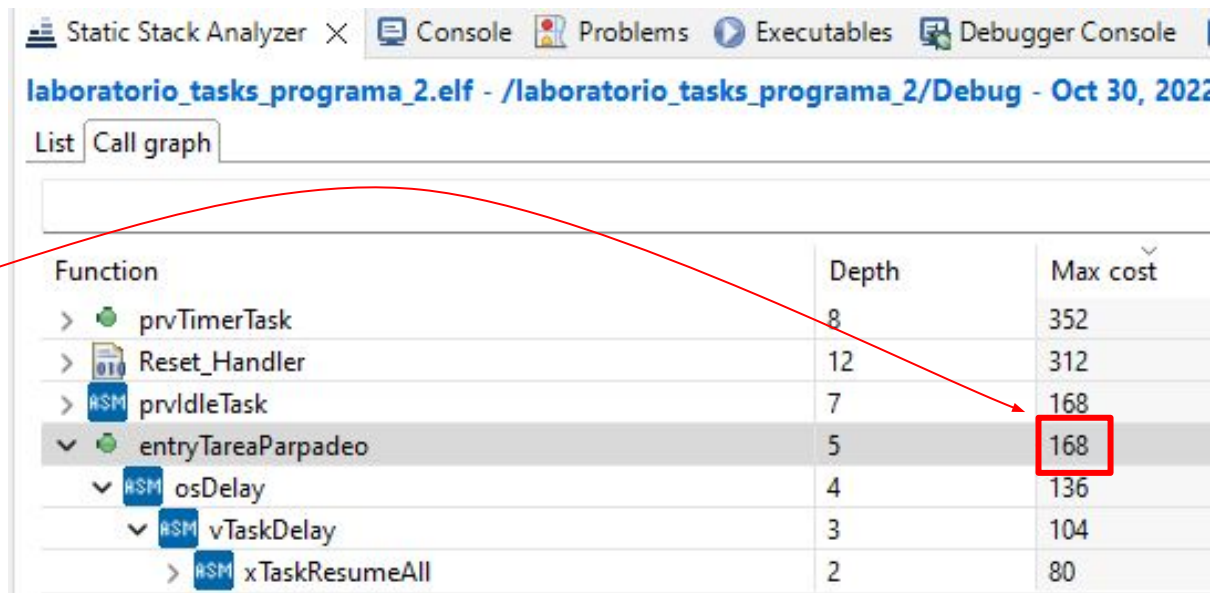
List Call graph

Function	Depth	Max cost
> prvTimerTask	8	352
> Reset_Handler	12	312
> prvIdleTask	7	168
▼ entryTareaParpadeo	5	168
▼ osDelay	4	136
▼ vTaskDelay	3	104
> xTaskResumeAll	2	80



Explorando el stack - En la compilación

- Para todas las funciones, analiza el “Call graph” que ve todos los posibles llamados de funciones y el “árbol” de llamadas que se desprende de cada una
- Devuelve el “costo máximo” que es el peor caso que logra estimar



Static Stack Analyzer × Console Problems Executables Debugger Console

laboratorio_tasks_programa_2.elf - /laboratorio_tasks_programa_2/Debug - Oct 30, 2023

List Call graph

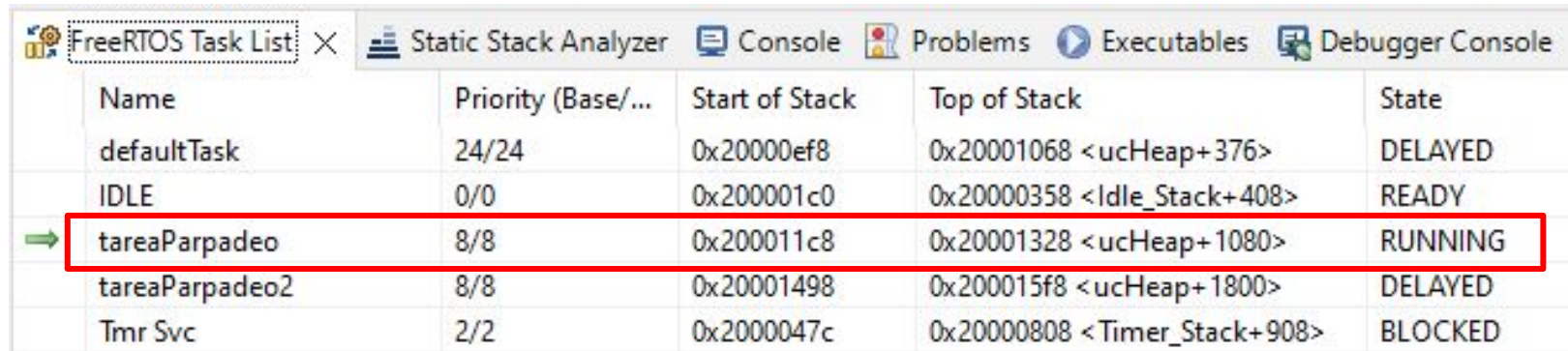
Function	Depth	Max cost
> prvTimerTask	8	352
> Reset_Handler	12	312
> prvIdleTask	7	168
✓ entryTareaParpadeo	5	168
✓ osDelay	4	136
✓ vTaskDelay	3	104
> xTaskResumeAll	2	80

Explorando el stack - En la ejecución

- Comprobemos la medición de máximo costo observando qué pasa con el stack de la tarea parpadeo durante la ejecución, deteniéndose con un breakpoint en un punto

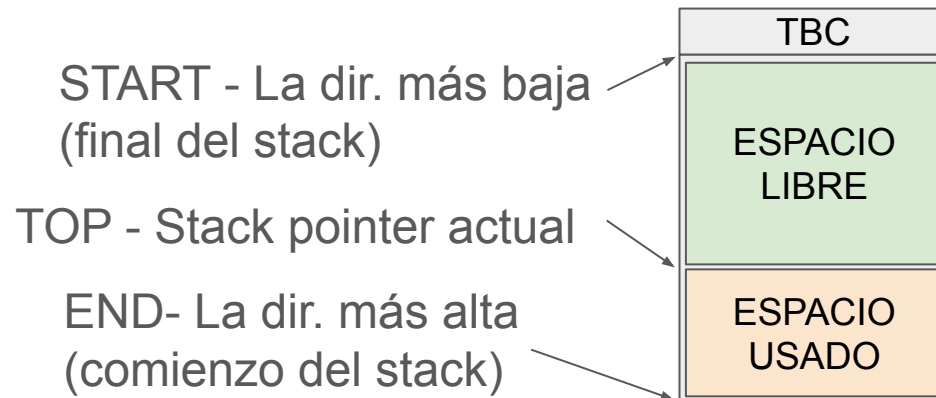
```
277 void entryTareaParpadeo(void *argument)
278 {
279     /* USER CODE BEGIN entryTareaParpadeo */
280
281     struct configParpadeo_t conf =
282         *((struct configParpadeo_t*)argument);
283     /* Infinite loop */
284     for(;;)
285     {
286         HAL_GPIO_TogglePin(conf.puerto, conf.pin);
287         osDelay(pdMS_TO_TICKS(conf.periodo));
288     }
289     /* USER CODE END entryTareaParpadeo */
290 }
```

Explorando el stack - En la ejecución



Name	Priority (Base/...)	Start of Stack	Top of Stack	State
defaultTask	24/24	0x20000ef8	0x20001068 <ucHeap+376>	DELAYED
IDLE	0/0	0x200001c0	0x20000358 <Idle_Stack+408>	READY
→ tareaParpadeo	8/8	0x200011c8	0x20001328 <ucHeap+1080>	RUNNING
tareaParpadeo2	8/8	0x20001498	0x200015f8 <ucHeap+1800>	DELAYED
Tmr Svc	2/2	0x2000047c	0x20000808 <Timer_Stack+908>	BLOCKED

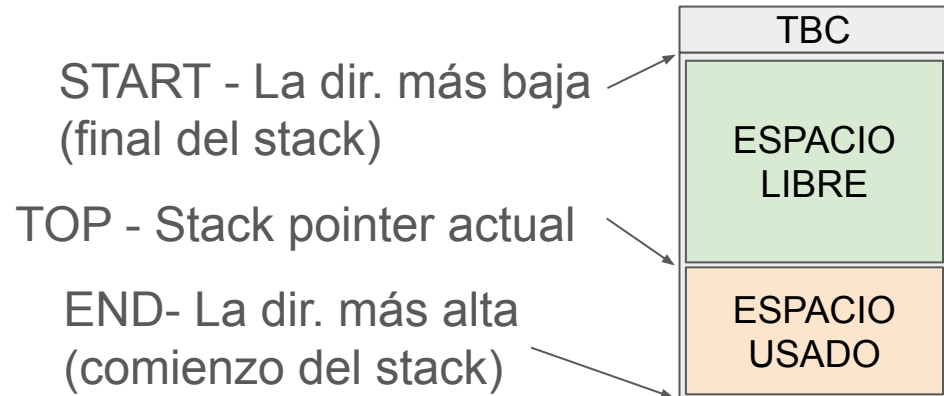
- Recordemos que el stack pointer es full-descending. Es decir que **arranca por la dirección más alta** y desciende hacia la más baja



Explorando el stack - En la ejecución

FreeRTOS Task List					
Name	Priority (Base/...	Start of Stack	Top of Stack	State	
defaultTask	24/24	0x20000ef8	0x20001068 <ucHeap+376>	DELAYED	
IDLE	0/0	0x200001c0	0x20000358 <Idle_Stack+408>	READY	
→ tareaParpadeo	8/8	0x200011c8	0x20001328 <ucHeap+1080>	RUNNING	
tareaParpadeo2	8/8	0x20001498	0x200015f8 <ucHeap+1800>	DELAYED	
Tmr Svc	2/2	0x2000047c	0x20000808 <Timer_Stack+908>	BLOCKED	

tareaParpadeo			
	byte (hex)	word (dec)	byte (dec)
Start	11c8		4552
Top	1328		4904
Depth	200	128	512
End	13c8		5064
Usado	A0	40	160



Explorando el stack

- Podemos visualizar la memoria RAM directamente para examinar la tarea

tareaParpadeo			
	byte (hex)	word (dec)	byte (dec)
Start	11c8		4552
Top	1328		4904
Depth	200	128	512
End	13C8		5064
Usado	A0	40	160

Memory × FreeRTOS Task List Static Stack Analyzer Console Problems Executa

Monitors + × 0x200011C8

0x200011C8 : 0x200011C8 <Hex> × + New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
200011C0	00000000	08020080	A5A5A5A5	A5A5A5A5
200011D0	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
200011E0	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
200011F0	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001200	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001210	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001220	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001230	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001240	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001250	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
...				
20001310	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001320	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001330	A5A5A5A5	68130020	A5A5A5A5	A5A5A5A5
20001340	A5A5A5A5	A5A5A5A5	00000000	D4130020
20001350	00000010	04ED00E0	A5A5A5A5	FD2B0008
20001360	DC290008	00000061	A5A5A5A5	F4010000
20001370	A5A5A5A5	00000000	80130020	911A0008
20001380	00080140	20000000	20002000	00080140
20001390	A5A5A5A5	20A00000	F00A0008	A0130020
200013A0	00000020	00000020	A5A5A5A5	00080140
200013B0	20000000	F4010000	A5A5A5A5	F53A0008
200013C0	A5A5A5A5	A5A5A5A5	00000000	C8000080

Explorando el stack

- Y un detalle con las variables locales:

var. conf de TareaParpadeo

Expression	Type	Value
&conf	struct configPa...	0x200013ac <u...
conf	struct configPa...	{...}
puerto	GPIO_TypeDef *	0x40010800
pin	uint32_t	32
periodo	uint32_t	500
Add new e		

Memory × FreeRTOS Task List Static Stack Analyzer Console Problems Executa

Monitors + × 0x200011C8

0x200011C8 : 0x200011C8 <Hex> × New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
200011C0	00000000	08020080	A5A5A5A5	A5A5A5A5
200011D0	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
200011E0	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
200011F0	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001200	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001210	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001220	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001230	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001240	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001250	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
...				
20001310	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001320	A5A5A5A5	A5A5A5A5	A5A5A5A5	A5A5A5A5
20001330	A5A5A5A5	68130020	A5A5A5A5	A5A5A5A5
20001340	A5A5A5A5	A5A5A5A5	00000000	D4130020
20001350	00000010	04ED00E0	A5A5A5A5	FD2B0008
20001360	DC290008	00000061	A5A5A5A5	F4010000
20001370	A5A5A5A5	00000000	80130020	911A0008
20001380	00080140	20000000	20002000	00080140
20001390	A5A5A5A5	20A00000	F00A0008	A0130020
200013A0	00000020	00000020	A5A5A5A5	00080140
200013B0	20000000	F4010000	A5A5A5A5	F53A0008
200013C0	A5A5A5A5	A5A5A5A5	00000000	C8000080

000001F4
00000020

Explorando el stack

- Y el TCB ?

watch expression `pxCurrentTCB`

✓ ➔ pxCurrentTCB	TCB_t * volatile	0x200013d0 <ucH
> ➔ pxTopOfStack	volatile StackType_t *	0x20001328 <ucH
> xStateListItem	ListItem_t	{...}
> xEventListItem	ListItem_t	{...}
(x)= uxPriority	UBaseType_t	8
> ➔ pxStack	StackType_t *	0x200011c8 <ucH
> pcTaskName	char [16]	0x20001404 <ucH
(x)= uxTCBNumber	UBaseType_t	2
(x)= uxTaskNumber	UBaseType_t	0
(x)= uxBasePriority	UBaseType_t	8
(x)= uxMutexesHeld	UBaseType_t	0

Monitor de memoria

Address	0 - 3	4 - 7	8 - B	C - F
200013C0	????	???	???	???
200013D0	(??	δ???	h?	h?
200013E0	θ??	`?	0???	0???
200013F0	0???	θ??	0???	0???
20001400	È??	tare	aPar	pade
20001410	o???	???	0???	0???
20001420	0???	0???	0E??	PE??
20001430	0E??	0???	0???	0???
20001440	0???	0???	0???	0???
20001450	0???	0???	0???	0???
20001460	0???	0???	0???	0???
20001470	0???	0???	0???	0???
20001480	0???	0???	0???	0???
20001490	0???	0 ??	????	????

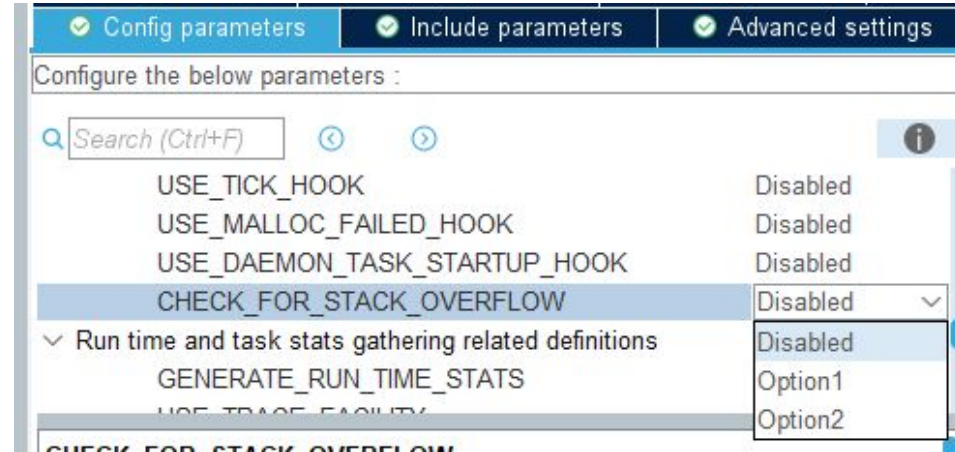
TBC

Comienzo de stack de la tarea

Más herramientas para detectar errores de stack

- Se puede configurar la detección de desborde de stack “automática”
- Opción 1: Chequea si el puntero de stack excede el frame asignado al momento del cambio de contexto
- Opción 2: Hace lo mismo que la opción 1, pero además chequea si los últimos bytes del stack siguen siendo “A5” o se sobrescribieron
- En caso de detectar desborde, se llama un callback:

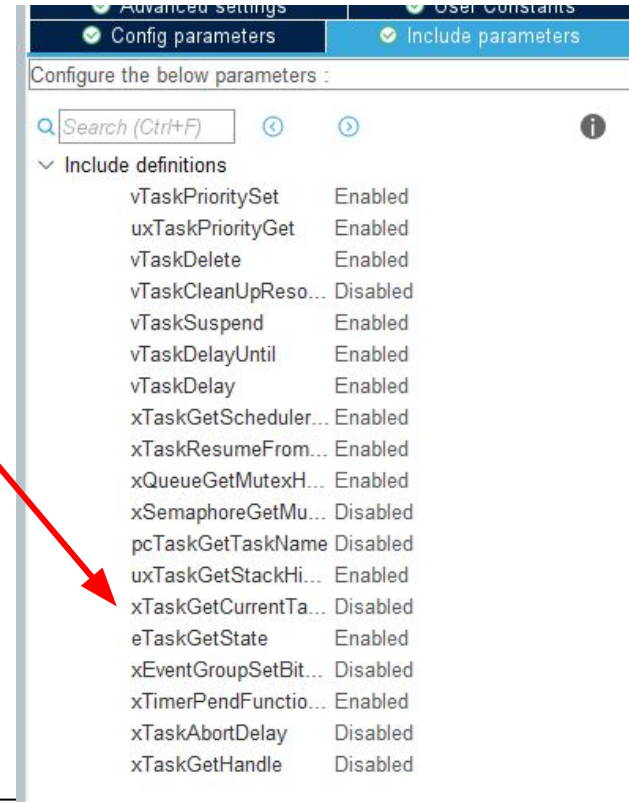
```
void vApplicationStackOverflowHook(  
xTaskHandle xTask, signed char *pcTaskName );
```



Más herramientas para detectar errores de stack

- En tiempo de ejecución el sistema puede obtener información de la cantidad de palabras libres en el stack invocando a la función **UBaseType_t uxTaskGetStackHighWaterMark(TaskHandle_t xTask);**
- INCLUDE_uxTaskGetStackHighWaterMark** tiene que estar seteado en 1 para que esta función esté disponible
- En CMSIS V2 se usa a través de la función :

```
uint32_t osThreadGetStackSpace (osThreadId_t thread_id) {  
    TaskHandle_t hTask = (TaskHandle_t)thread_id;  
    uint32_t sz;  
  
    if (IS_IRQ() || (hTask == NULL)) {  
        sz = 0U;  
    } else {  
        sz = (uint32_t)uxTaskGetStackHighWaterMark (hTask);  
    }  
}
```



Vemos un ejemplo:

Tasks			
Task Name	Pr...	Stack Siz...	Entry Function
defaultTask	o...	128	StartDefaultTask
myTaskLectura	o...	128	entryTaskLectura
myTaskControlad	o...	128	entryTaskControlada

```
void entryTaskLectura(void *argument) {
    /* USER CODE BEGIN entryTaskLectura */
    /* Infinite loop */
    for (;;) {
        libres_lectura =
            4*osThreadGetStackSize(myTaskLecturaHandle);
        libres_controlada =
            4*osThreadGetStackSize(myTaskControladHandle);
        libres_heap = xPortGetFreeHeapSize();
        osDelay(1000);
    }
    /* USER CODE END entryTaskLectura */
}
```

```
/* USER CODE BEGIN PV */
uint32_t libres_lectura;
uint32_t libres_controlada;
uint32_t libres_heap;
/* USER CODE END PV */
```

```
void entryTaskControlada(void *argument) {
    /* USER CODE BEGIN entryTaskControlada */
    /* Infinite loop */
    int n = 0;
    for (;;) {
        n += 10;
        func(n);
        osDelay(2000);
    }
    /* USER CODE END entryTaskControlada */
}
```

Vemos un ejemplo:

```
void entryTaskControlada(void *argument) {
    /* USER CODE BEGIN entryTaskControlada */
    /* Infinite loop */
    int n = 0;
    for (;;) {
        n += 10;
        func(n);
        osDelay(2000);
    }
    /* USER CODE END entryTaskControlada */
}
```

```
/* USER CODE BEGIN 4 */
void func(int n) {
    int arr[n];

    int i;
    for (i = 0; i < n; i++) {
        arr[i] = 10;
    }
    return;
}
```

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlada	uint32_t	352
(x)= libres_heap	uint32_t	1192

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlada	uint32_t	152
(x)= libres_heap	uint32_t	1192

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlada	uint32_t	32
(x)= libres_heap	uint32_t	1192

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlada	uint32_t	0
(x)= libres_heap	uint32_t	1192

```
83  * @brief This function handles HardFault interrupt
84  */
85  void HardFault_Handler(void)
86  {
87      /* USER CODE BEGIN HardFault_IRQn 0 */
88
89      /* USER CODE END HardFault_IRQn 0 */
90      while (1)
91      {
92          /* USER CODE BEGIN W1_HardFault_IRQn 0 */
93          /* USER CODE END W1_HardFault_IRQn 0 */
94      }
95  }
```

Usando malloc

```
void entryTaskControlada(void *argument) {
    /* USER CODE BEGIN entryTaskControlada */
    /* Infinite loop */
    int n = 0;
    for (;;) {
        n += 10;
        func(n);
        osDelay(2000);
    }
    /* USER CODE END entryTaskControlada */
}
```

```
void func(int n) {

    int *arr = (int*) pvPortMalloc(sizeof(int)*n);
    int i;
    for (i = 0; i < n; i++) {
        arr[i] = 10;
    }
    return;
}
```

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlac	uint32_t	372
(x)= libres_heap	uint32_t	1144

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlac	uint32_t	372
(x)= libres_heap	uint32_t	928

Expression	Type	Value
(x)= libres_lectura	uint32_t	360
(x)= libres_controlac	uint32_t	372
(x)= libres_heap	uint32_t	0

```
85 void HardFault_Handler(void)
86 {
87     /* USER CODE BEGIN HardFault_IRQn 0 */
88
89     /* USER CODE END HardFault_IRQn 0 */
90     while (1)
91     {
```

Malloc permite detectar fallos

```
void entryTaskControlada(void *argument) {  
    /* USER CODE BEGIN entryTaskControlada */  
    /* Infinite loop */  
    int n = 0;  
    for (;;) {  
        n += 10;  
        func(n);  
        osDelay(2000);  
    }  
    /* USER CODE END entryTaskControlada */  
}
```

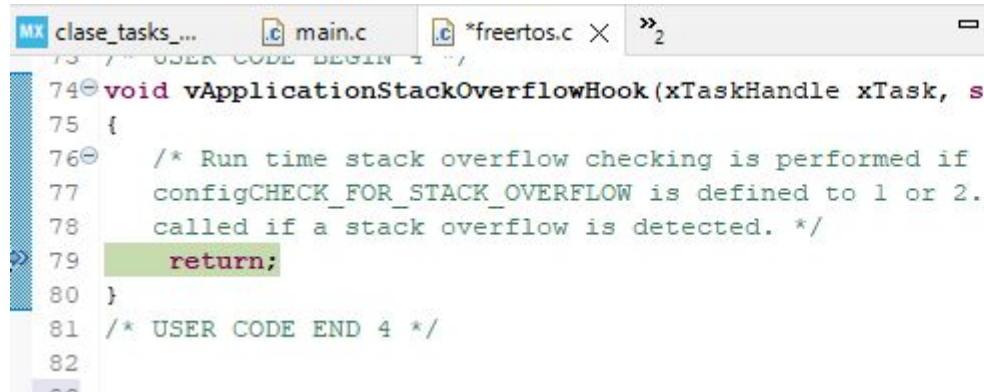
```
228 /* USER CODE BEGIN 4 */  
229 void func(int n) {  
230  
231     int *arr = (int*) pvPortMalloc(sizeof(int)*n);  
232     int i;  
233  
234     if(arr == NULL ) {  
235         return;  
236     }
```

Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlad	uint32_t	372
(x)= libres_heap	uint32_t	0
+ Add new expre:		

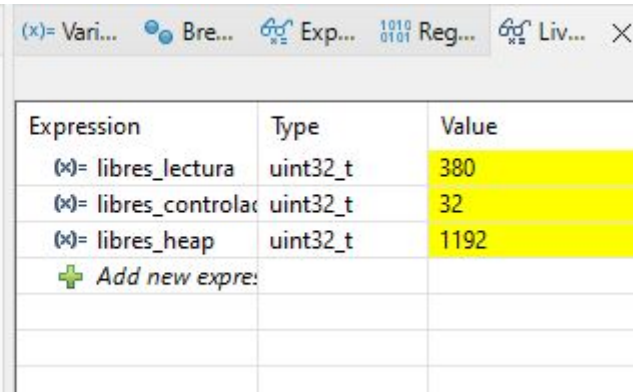
Volviendo al ejemplo de stack overflow

```
void entryTaskControlada(void *argument) {  
    /* USER CODE BEGIN entryTaskControlada */  
    /* Infinite loop */  
    int n = 0;  
    for (;;) {  
        n += 10;  
        func(n);  
        osDelay(2000);  
    }  
    /* USER CODE END entryTaskControlada */  
}
```

```
/* USER CODE BEGIN 4 */  
void func(int n) {  
    int arr[n];  
  
    int i;  
    for (i = 0; i < n; i++) {  
        arr[i] = 10;  
    }  
    return;  
}
```



```
74 void vApplicationStackOverflowHook(xTaskHandle xTask, s  
75 {  
76     /* Run time stack overflow checking is performed if  
77     configCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2.  
78     called if a stack overflow is detected. */  
79     return;  
80 }  
81 /* USER CODE END 4 */  
82
```



Expression	Type	Value
(x)= libres_lectura	uint32_t	380
(x)= libres_controlada	uint32_t	32
(x)= libres_heap	uint32_t	1192
+ Add new expres...		

Prioridades de las tareas

Prioridades

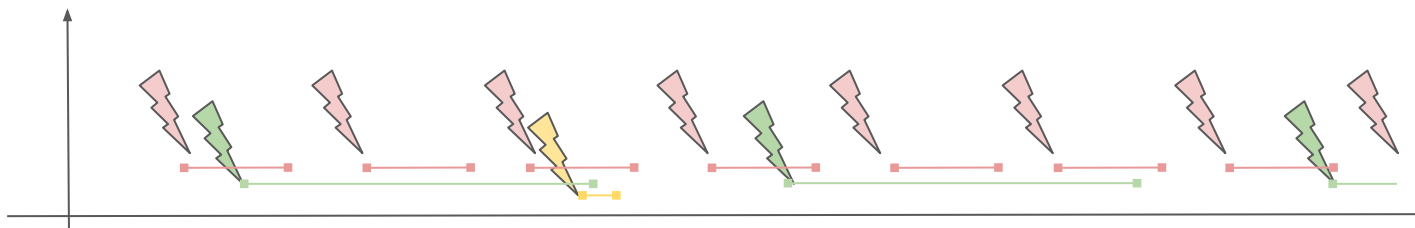
- CMSIS-RTOS v2, tiene un esquema de prioridades con 8 niveles, cada uno a su vez subdividido en 8
- Esos niveles se mapean a prioridades de FreeRTOS según se encuentra en cmsis_os2.h
- `osThreadSetPriority()` permite cambiar la prioridad de un thread y `osThreadGetPriority()` obtener su valor
- La función que ya comentamos, `osThreadYield()`, sólo cederá la ejecución a una tarea de mayor prioridad, pero no a una de menor prioridad aunque la misma esté READY

```
/// Priority values.
typedef enum {
    osPriorityNone           = 0,
    osPriorityIdle           = 1,
    osPriorityLow            = 8,
    osPriorityLow1          = 8+1,
    ...
    osPriorityLow7          = 8+7,
    osPriorityBelowNormal   = 16,
    ...
    osPriorityBelowNormal7  = 16+7,
    osPriorityNormal        = 24,
    ...
    osPriorityNormal7       = 24+7,
    osPriorityAboveNormal   = 32,
    ...
    osPriorityAboveNormal7  = 32+7,
    osPriorityHigh          = 40,
    ...
    osPriorityHigh7         = 40+7,
    osPriorityRealtime       = 48,
    ...
    osPriorityRealtime7     = 48+7,
    osPriorityISR            = 56,
    osPriorityError         = -1,
} osPriority_t;
```


Eventos e Interrupciones

Atención de eventos

- Al trabajar con sistemas embebidos de tiempo real lidiamos con la respuesta en tiempos acotados a eventos (vimos latencia y jitter)
- Estos eventos pueden ser comunicados a través de periféricos o disparados por software
- La forma de notificar al sistema de su ocurrencia puede ser por interrupciones o por polling
- Podemos usar tareas e interrupciones como herramienta para programar la respuesta a eventos. ¿Cuánto hay que hacer en la tarea, y cuanto en el cuerpo de la interrupción? ¿Cómo se pueden comunicar entre sí? ¿Cómo organizar la ejecución?



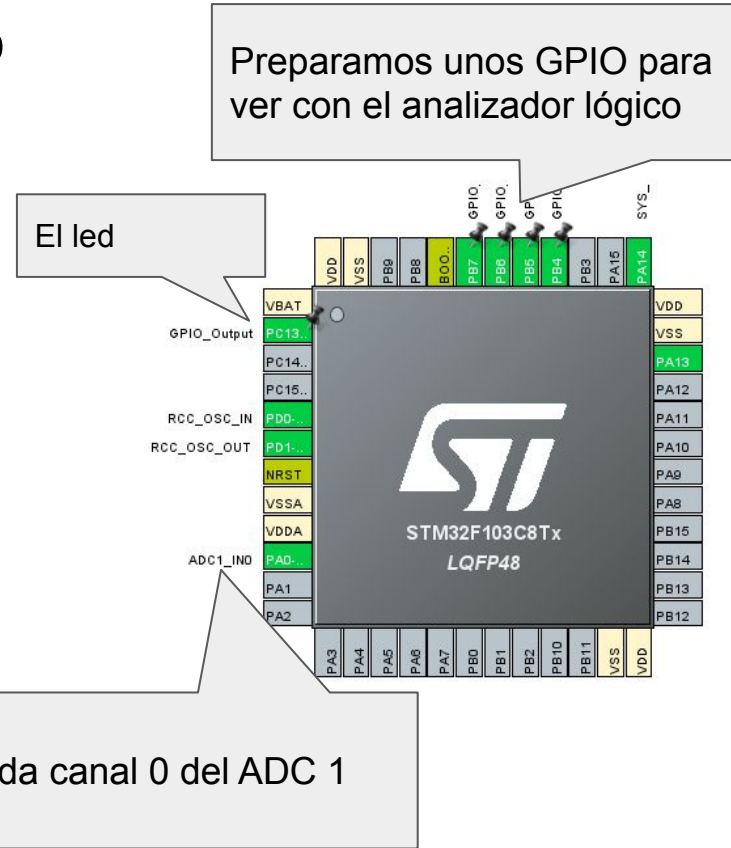
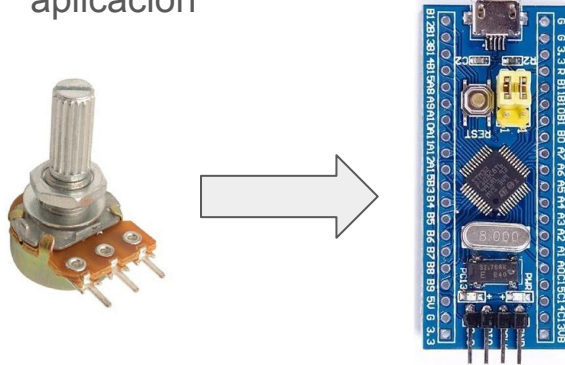
Interrupciones y prioridades

- Recordemos que las tareas tienen prioridades, y las interrupciones también tienen prioridades
- Las prioridades de las tareas le dicen al scheduler cuál poner a correr a continuación, dentro de un esquema de software que no está relacionada con el hardware
- Las interrupciones tienen una prioridad con efecto o resolución a nivel del Hardware (Procesador, NVIC y otros periféricos)
- Sin embargo, **ambos “mundos” se relacionan ya que el sistema operativo realiza el cambio de contexto a partir del uso de interrupciones.**



Volvemos a prioridades: ejemplo

- Se tiene una señal de entrada que simularemos con un potenciómetro
- Se quiere adquirir a una tasa de 20kHz (50 μ s de período) para obtener una tasa final de 2 kHz realizando un promedio de 10 muestras
- Se usa FreeRTOS en paralelo a esta aplicación



Preparamos la aplicación

Instalación de Tareas

Tasks and Queues Timers and Semaphores

Config parameters Include parameters

Tasks

Task Name	Priority	Sta...	Entry Function
TaskNormal	osPriorityNormal	128	entryTaskNormal
TaskAlta	osPriorityHigh	128	entryTaskAlta

Configuración de interrupción timer 2

RTC
TIM1
TIM2
TIM3
TIM4

Connectivity >

Parameter Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value) 71

Counter Mode Up

Counter Period (AutoReload ... 49

Internal Clock Division (CKD) No Division

auto-reload preload Enable

Timers

RTC
TIM1
TIM2
TIM3
TIM4

User Constants NVIC S

Parameter

NVIC Interrupt Table Enabled

TIM2 global interrupt ☒

Preparamos la aplicación

Debug con TaskHooks

```
/* FreeRTOSConfig.h*/  
void callback_in(int);  
void callback_out(int);  
#define traceTASK_SWITCHED_IN() callback_in((int)pxCurrentTCB->pxTaskTag)  
#define traceTASK_SWITCHED_OUT() callback_out((int)pxCurrentTCB->pxTaskTag)  
#define TAG_TASK_IDLE 0  
#define TAG_TASK_NORMAL 1  
#define TAG_TASK_ALTA 2
```

```
/* En cada tarea (incluir task.h)*/  
vTaskSetApplicationTaskTag(NULL, (void*) TAG_TASK_NORMAL);
```

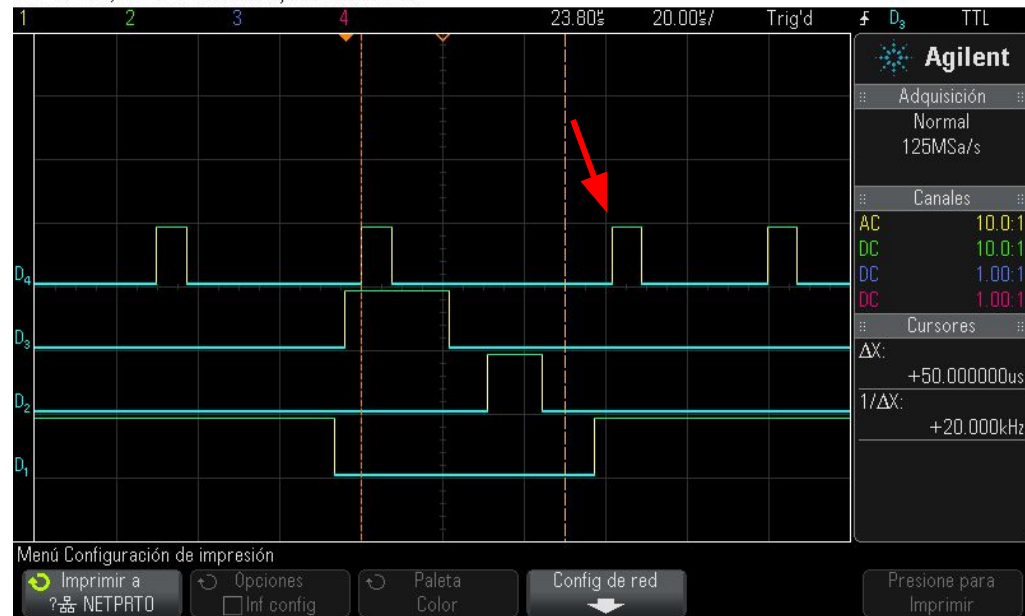
```
void callback_in(int tag) { /* Definición en main.c*/  
    switch (tag) {  
        case TAG_TASK_IDLE: HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);break;  
        case TAG_TASK_NORMAL: HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);break;  
        case TAG_TASK_ALTA: HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);break;  
    }  
}
```

Preparamos la aplicación

En la interrupción del timer disparamos la conversión y tomamos la muestra

```
void HAL_TIM_PeriodElapsedCallbackTIM2 (TIM_HandleTypeDef *htim) {  
    if (htim->Instance == TIM2) {  
        HAL_GPIO_WritePin (GPIOB, GPIO_PIN_7, GPIO_PIN_SET);  
        HAL_ADC_Start (&hadc1);  
        HAL_ADC_PollForConversion (&hadc1, 100);  
        val = HAL_ADC_GetValue (&hadc1);  
        HAL_GPIO_WritePin (GPIOB, GPIO_PIN_7, GPIO_PIN_RESET);  
    }  
}
```


Veamos la salida:



En este caso, el esquema de prioridades ocasionó un incumplimiento del período.
Vamos a analizar por qué

Niveles de prioridad

Interrupt	Enabled	Preemption Priority	Sub Priority	Uses FreeRTOS functions
Non maskable interrupt	☒	0	0	☐
Hard fault interrupt	☒	0	0	☐
Memory management fault	☒	0	0	☐
Prefetch fault, memory access fault	☒	0	0	☐
Undefined instruction or illegal state	☒	0	0	☐
System service call via SWI instruction	☒	0	0	☐
Debug monitor	☒	0	0	☐
Pendable request for system service	☒	15	0	☒
System tick timer	☒	15	0	☒
PVD interrupt through EXTI line 16	☐	5	0	☒
Flash global interrupt	☐	5	0	☒
RCC global interrupt	☐	5	0	☒
ADC1 and ADC2 global interrupts	☐	5	0	☒
TIM2 global interrupt	☒	5	0	☒
Time base: TIM4 global interrupt	☒	5	0	☐

☒ Enabled Preemption Priority: 5 ☒ Uses FreeRTOS functions

(3) Esos valores son configurables, y la limitación sobre el nivel de la interrupción se debe a una configuración activada por default

(1) La prioridad de interrupción del timer está limitada en 5 en el entorno gráfico

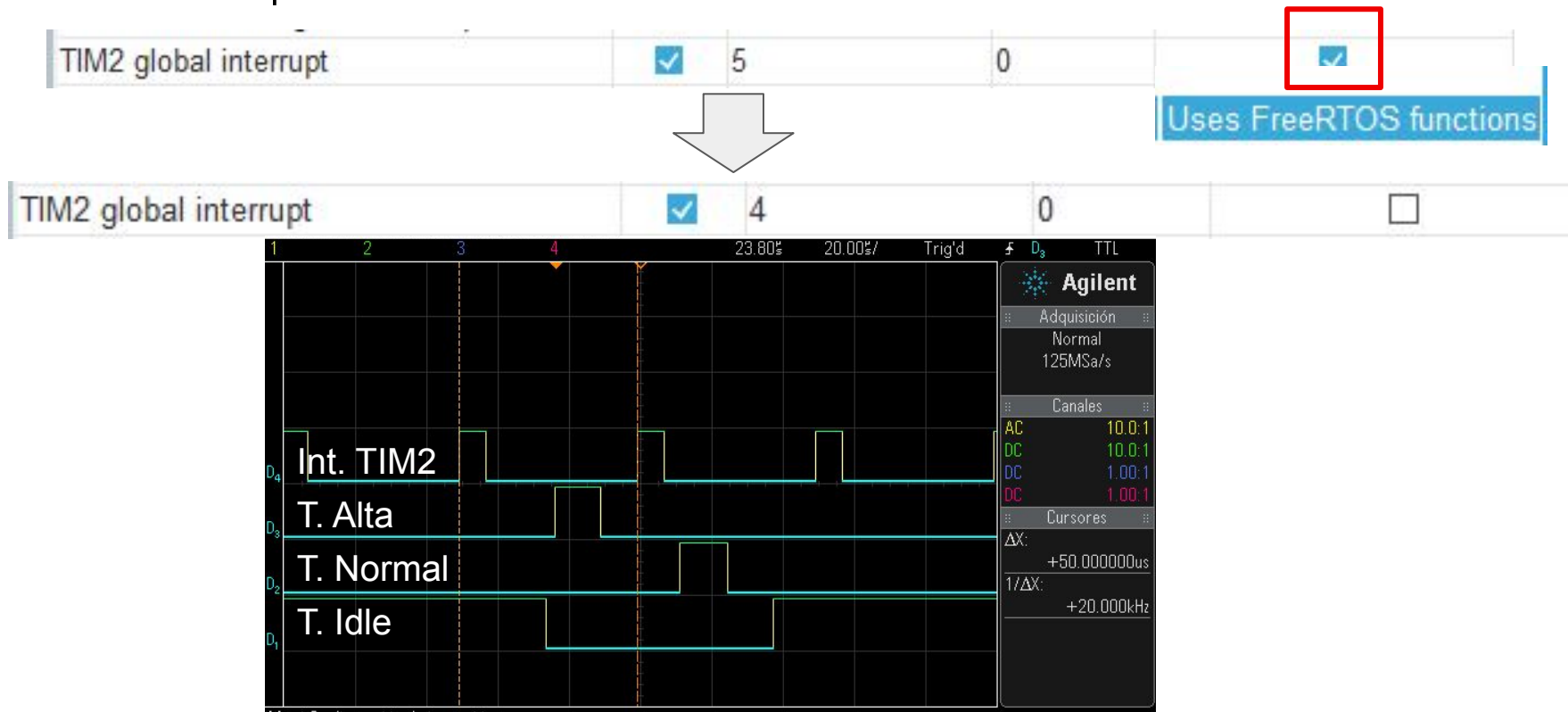
(2) Es el mismo nivel de prioridad del PVD que se llama para producir un cambio de contexto

Niveles de prioridad

```
▼ Interrupt nesting behaviour configuration
    LIBRARY_LOWEST_INTERRUPT_PRIORITY      15
    LIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY  5
```

- **LIBRARY_LOWEST_INTERRUPT_PRIORITY**
La prioridad más baja asignable (siguiendo en este caso la sintaxis del Cortex M3 con 16 niveles)
- **LIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY**
La mayor prioridad que se puede asignar que va a seguir estando dentro del esquema de funcionamiento del sistema operativo, y por lo tanto va a **permitir llamar funciones de la API compatibles con interrupciones**. Si se asigna una prioridad mayor (número más bajo) ya no podrán usarse funciones de la API
- FreeRTOS suspende interrupciones para asegurar la coherencia interna de sus funciones, por eso no podemos usar la API de FreeRTOS si estamos por encima del nivel máximo que maneja, podríamos producir problemas de concurrencia.

Niveles de prioridad



Niveles de prioridad

- Resolvimos la especificación de tiempo real al disparar la conversión en el tiempo adecuado
- Pero ahora queremos enviar el dato capturado a una tarea para ser procesado, y nos encontramos que estamos fuera del RTOS
- Podemos distribuir de otra manera el resto del procesamiento.
- Por ejemplo, podemos usar la interrupción del ADC, pero sin aumentarle tanto la prioridad



ADC1 and ADC2 global interrupts	☒	5
TIM2 global interrupt	☒	4

Niveles de prioridad

```
void HAL_TIM_PeriodElapsedCallbackTIM2(TIM_HandleTypeDef *htim) {  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_SET);  
    HAL_ADC_Start_IT(&hadc1);  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET);  
}
```

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);  
    int16_t val;  
    val = HAL_ADC_GetValue(&hadc1);  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);  
}
```

ADC1 and ADC2 global interrupts	☒	5
TIM2 global interrupt	☒	4

Niveles de prioridad



ADC1 and ADC2 global interrupts



5

TIM2 global interrupt

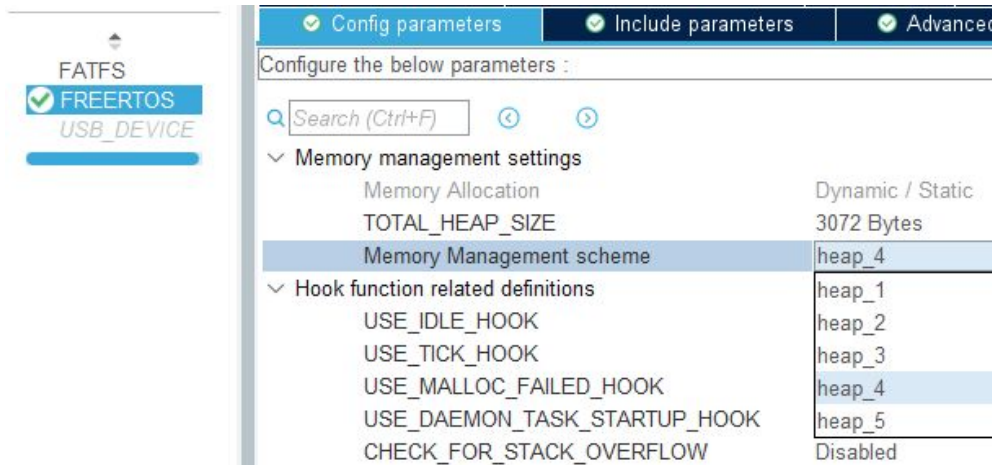


4

Manejo de Memoria

Asignación de memoria

- Las tareas pueden crearse y borrarse dinámicamente, según el esquema de asignación de memoria elegido
- ¿Quién se encarga de asignar memoria?
- ¿Y si, como suele suceder, necesitamos asignar memoria dinámica para nuestras variables?
- Existen distintos esquemas que pueden elegirse



Asignación de memoria

- **heap_1.c**

- Es el esquema más simple y no permite que se libere memoria una vez que se asignó.
- Subdivide un único arreglo en bloques cuando se realizan pedidos de espacio en RAM
- El tamaño total del arreglo se define a partir de configTOTAL_HEAP_SIZE en FreeRTOSConfig.h.
- Se puede utilizar si no se eliminan elementos (no hay llamadas a vTaskDelete () o vQueueDelete () por ejemplo)
- Siempre es determinista: toma siempre el mismo tiempo para reservar un bloque.

- **heap_2.c**

- Este esquema busca entre la memoria libre y elige el bloque que mejor se ajusta al tamaño necesario (best-fit).
- Además, puede liberar bloques (permite usar vTaskDelete() y vQueueDelete())
- Sin embargo, no puede juntar bloques que se habían reservado previamente aunque se liberen, lo cual puede conducir a la fragmentación de la memoria.
- El tamaño total de RAM disponible se configura con configTOTAL_HEAP_SIZE
- No es determinista ya que depende del algoritmo que encuentra el mejor bloque disponible, pero al ser un algoritmo sencillo pueden predecirse tiempos máximos con cotas optimistas.

Asignación de memoria

- **heap_4.c**

- El algoritmo asigna el primer bloque suficientemente grande para la memoria necesaria, combinando bloques libres adyacentes
- Es conveniente cuando se repite la creación y borrado de tareas y queues, pero también cuando se requieren numerosos llamados a vPortMalloc() and vPortFree()
- No es determinista, pero vPortMalloc/Free son más eficientes que malloc/free estándar de las librerías de C
- Por este conjunto de características, es el algoritmo seleccionado por defecto

- **Asignación estática**

- En lugar de algoritmos dinámicos puede seleccionarse la opción estática y se asignan bloques de un arreglo programando las direcciones.

Elementos de sincronización en el RTOS

Semáforos

Coordinación entre Tareas y/o Interrupciones

- Existen distintas herramientas para coordinar las tareas entre sí, y las interrupciones con las tareas; la provisión de estos mecanismos es una de las razones para usar el OS
- Veamos la más sencilla: los semáforos, una herramienta de sincronización

semáforo

Del gr. σῆμα *sêma* 'señal' y -foro.

1. m. Aparato eléctrico de señales luminosas para regular la circulación.
2. m. Telégrafo óptico de las costas, para comunicarse con los buques por medio de señales.
3. m. Cualquier sistema de señales ópticas. *Semáforo de banderas.*



Semáforo

- Puede pensarse como una variable que se puede decrementar con una función **Take, Recibir o Adquirir** o incrementarse con una **Give, Dar, Soltar, Release**, y que **nunca puede valer menos de cero**
- Alternativamente (explicación de R. Barry) puede pensarse como una Queue de la cual no nos interesa los valores que almacena, sólo guardamos y recuperamos valores como forma de comunicación

```
var = 0;

funcion tomarSemaforo
    SI var>0
        var = var -1
    SI NO
        bloquear tarea
    FIN SI
fin funcion

funcion darSemaforo
    var = var + 1
fin funcion
```

Semáforo

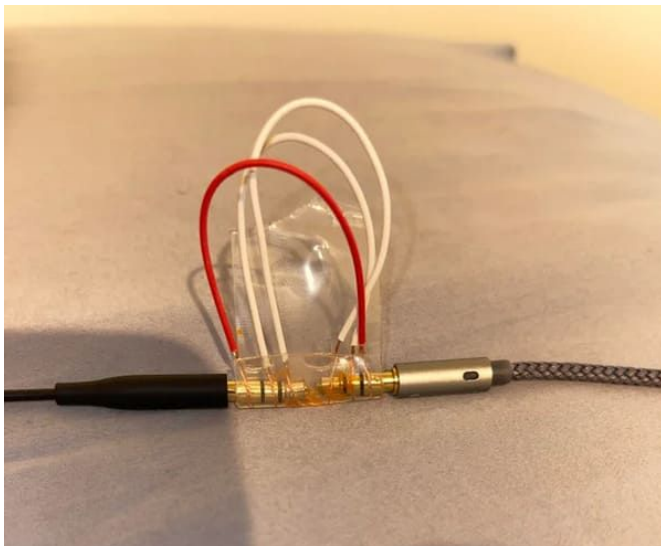
- La utilidad consiste en
 - Contar eventos: damos el semáforo cada vez que ocurre un evento de interés (e.g. en cada conversión del ADC) y luego vemos su valor
 - Administrar recursos: Es similar a contar eventos, por ejemplo en protocolos de comunicación complejos donde puede haber varios “clientes”, pero no infinitos, puede haber un semáforo que sólo pueda tomarse N veces
 - ¿Y qué lo diferencia de una variable común? Que es un elemento de sincronización con **operaciones atómicas**: no tendremos problemas de concurrencia en el acceso
- El semáforo binario
 - Es una herramienta sencilla para coordinar tareas, es un semáforo que **puede contar sólo hasta 1**
 - Se usa para permitir a una tarea avanzar a partir de cierto punto si se cumple un cierto evento. En FreeRTOS se distingue de los semáforos comunes por nombre

```
var = 0;
```

```
funcion tomarSemaforoBin  
    SI var>0  
        var = var -1  
    SI NO  
        bloquear tarea  
    FIN SI  
fin funcion
```

```
funcion darSemaforoBin  
    var = 1  
fin funcion
```


Segmento: ¡Hágalo usted mismo!



Tarea 1

```
for(;;) {  
    flag_evt++; // Se señala nuevo evento  
    osDelay(1000);  
}
```

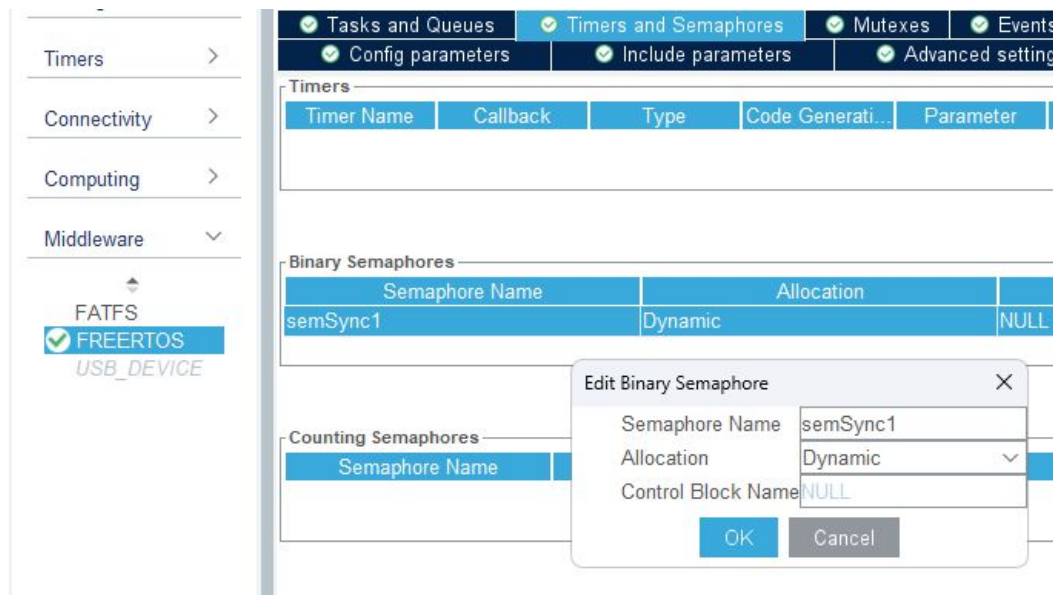
Tarea 2

```
for(;;) {  
    if( flag_evt > 0){ // Chequea flag  
        flag_evt = flag_evt-1; // Se descuenta el evento  
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_3);  
    }  
    else{  
        osThreadYield(); // Si no, se cede la ejecución  
    }  
}
```

Implementar el código e investigar el potencial problema de concurrencia.

Ejemplo de sincronización

- Primero, se crea un semáforo binario en el IDE



Ejemplo de sincronización

- Y lo usamos para señalar, por ejemplo, cuando se han leído 10 valores

```
/* USER CODE END Header_entryTaskAlta */
void entryTaskAlta(void *argument)
{
    /* USER CODE BEGIN entryTaskAlta */
    vTaskSetApplicationTaskTag( NULL, (void*) TAG_TASK_ALTA);
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    HAL_TIM_Base_Start_IT(&htim2);
    count = 0;
    for (;;) {
        /*La tarea se bloqueará a partir de la próxima línea*/
        osSemaphoreAcquire(semSync1Handle, osWaitForever);
        /* Cuando se libere, reseteará la variable */
        count = 0;
        osDelay(1);
    }
    /* USER CODE END entryTaskAlta */
}
```

```
void HAL_ADC_ConvCpltCallback (ADC_HandleTypeDef * hadc)
{
    HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6,
    GPIO_PIN_SET);
    val = HAL_ADC_GetValue (&hadc1);
    count++;
    if (count== 10){
        osSemaphoreRelease (semSync1Handle);
    }
    HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6,
    GPIO_PIN_RESET);
}
```

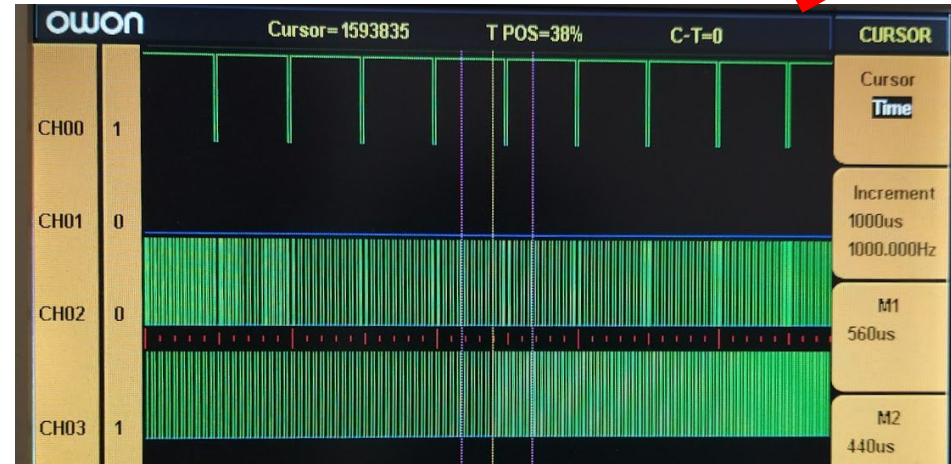
Ejemplo de sincronización



↑ Cada 10 valores se desbloquea la tarea

Al no dar el semáforo, la tarea nunca puede tomarlo →

```
void HAL_ADC_ConvCpltCallback (ADC_HandleTypeDef *hadc) {
    val = HAL_ADC_GetValue (&hadc1);
    count++;
    if(count== 10){
        //osSemaphoreRelease(semSync1Handle);
    }
}
```



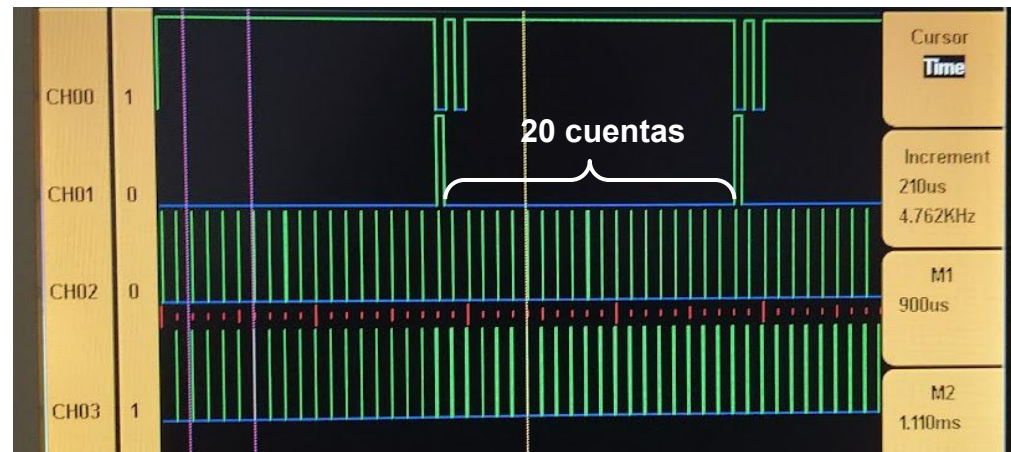
(Paréntesis: Funciones FromISR)

- FreeRTOS tiene una API separada para ser llamada desde las rutinas de interrupción
- Nunca deben llamarse las funciones que no son “seguras para ISR” (i.e. que terminan en FromISR) desde una interrupción
- Pero todos los elementos en general cuentan con sus APIs compatibles con interrupciones
- CMSIS v2 nos esconde esto y lo resuelve

```
osStatus_t osSemaphoreRelease (osSemaphoreId_t semaphore_id) {  
    SemaphoreHandle_t hSemaphore = ( SemaphoreHandle_t ) semaphore_id;  
    ...  
    if (hSemaphore == NULL) {  
        stat = osErrorParameter;  
    }  
    else if (IS_IRQ()) {  
        ...  
        xSemaphoreGiveFromISR (hSemaphore, &yield)  
    }  
    else {  
        if (xSemaphoreGive (hSemaphore) != pdPASS) {  
            stat = osErrorResource;  
        }  
    }  
  
    return (stat);  
}
```

Ejemplo de sincronización

- Cada vez que `count==10` se “libera” la ejecución de la tarea
- Pero esto no significa que la tarea se ejecute
- Sólo significa que cuando el scheduler la venga a buscar, la encontrará en estado READY y ya no bloqueada

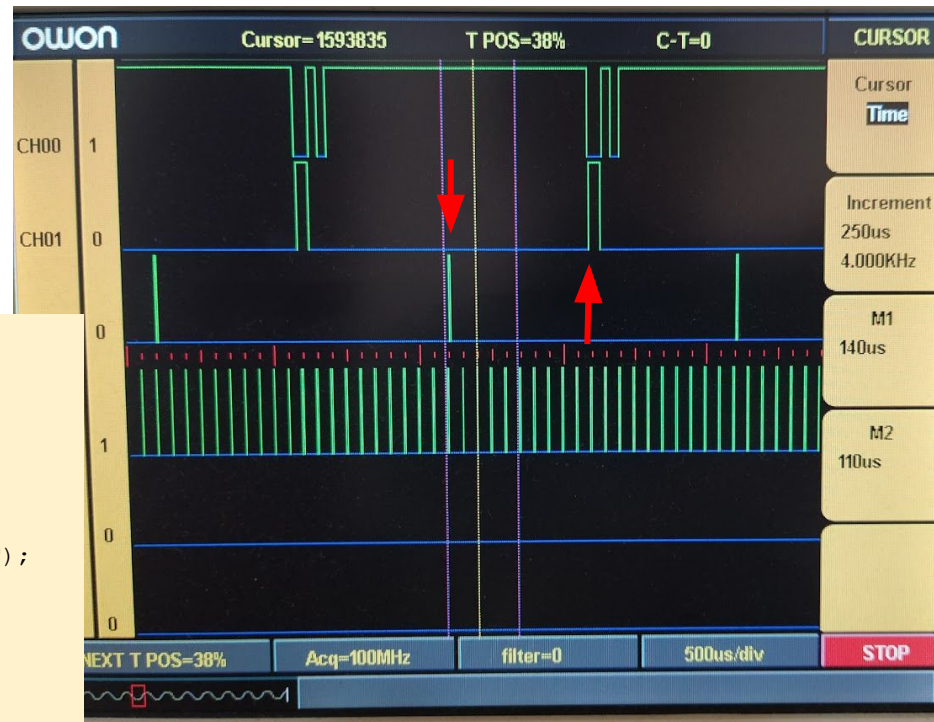


Live Expressions		
Expressions (x)= Variables Breakpoints		
Expression	Type	Value
prom		Failed to evaluate expression
std		Failed to evaluate expression
(x)= val	int16_t	1026
(x)= count	int	14
+ Add new expression		

Ejemplo de sincronización

- Puede observarse con un cambio menor en la visualización
- En este caso, si tenemos un buffer de 10 muestras, se desbordaría... ¿podemos “invocar” a la tarea?

```
void HAL_ADC_ConvCpltCallback (ADC_HandleTypeDef * hadc) {  
    //HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);  
    val = HAL_ADC_GetValue (&hadc1);  
    count++;  
    if(count== 10){  
        HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6, GPIO_PIN_SET);  
        osSemaphoreRelease (semSync1Handle);  
    }  
    HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);  
}
```



portYIELD_FROM_ISR()

- taskYIELD() se puede invocar dentro de una tarea para pedir un cambio de contexto a otra de mayor prioridad
- **portYIELD_FROM_ISR()** es una versión para **usar desde interrupciones** y permitirá que al terminar una interrupción se pida al scheduler un cambio de contexto
- El scheduler realizará el cambio **si hay una tarea de alta prioridad en estado Ready**
- La función toma un parámetro que por convención dentro de FreeRTOS se nombra **xHigherPriorityTaskWoken**

```
BaseType_t xHigherPriorityTaskWoken;  
xHigherPriorityTaskWoken = pdFALSE;  
xHigherPriorityTaskWoken = pdTRUE;
```

- Si el parámetro xHigherPriorityTaskWoken pasado a portYIELD_FROM_ISR() es pdFALSE, entonces la función (es un macro en realidad) no pide ningún cambio de contexto. **Si es pdTRUE se pide un cambio de contexto** y puede cambiar la tarea RUNNING
- portYIELD_FROM_ISR() Debe llamarse al final de la rutina de interrupción

xHigherPriorityTaskWoken

- Las funciones de la API de FreeRTOS usan este parámetro y se le puede pasar con el valor pdTRUE para indicar que luego de hacer lo que hagan (e.g. entregar un semáforo) se pide un cambio de contexto
- Pero CMSIS v2 al manipular los llamados en una API que sirve tanto para tareas como para ISRs “toquetea” este parámetro
- Podemos usar el macro PortYield directamente

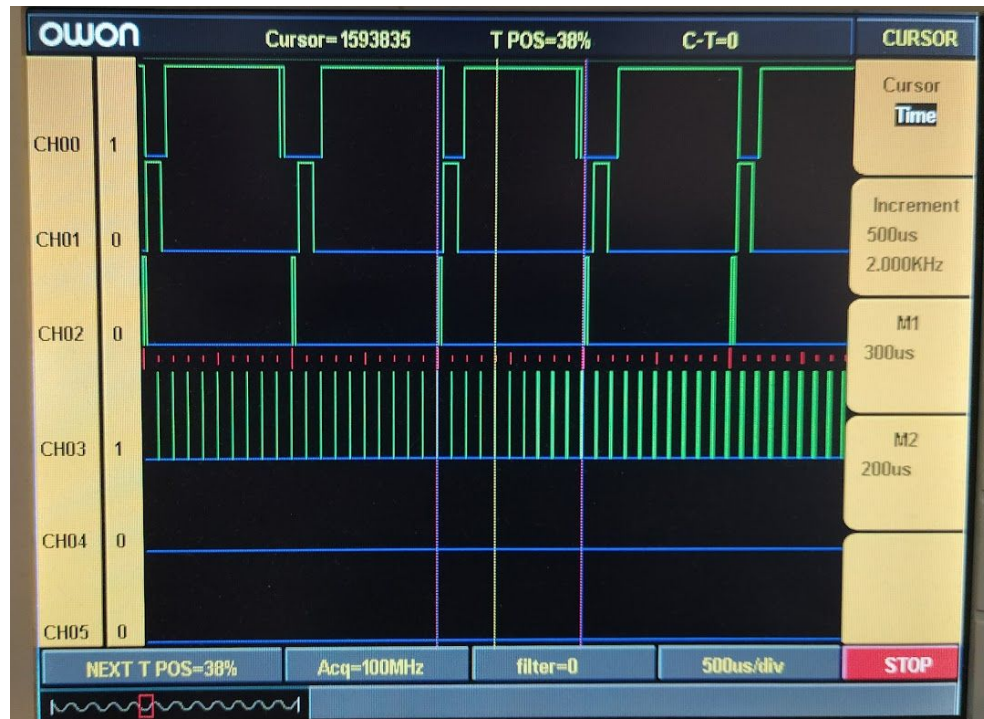
xHigherPriorityTaskWoken

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {  
    BaseType_t xHigherPriorityTaskWoken = pdFALSE; ←  
    val = HAL_ADC_GetValue(&hadc1);  
    count++;  
    if(count== 10){  
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);  
        xHigherPriorityTaskWoken = pdTRUE; ←  
        osSemaphoreRelease(semSynclHandle);  
    }  
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);  
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken ); ←  
}
```

Ejemplo de sincronización



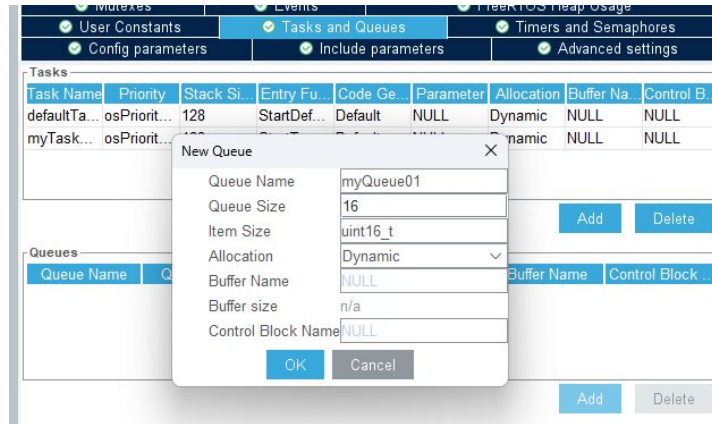
antes y después



Colas

Colas en FreeRTOS

- Una cola o Queue puede contener un número finito de ítems de tamaño fijo que se configuran al momento de su creación.
- El tamaño máximo que puede contener es su largo o tamaño (length, size)



- Son buffers FIFO y al escribir un dato el mismo se copia al buffer que mantiene la cola. Al leer, el dato se borra.

Colas en FreeRTOS

- La API de CMSIS V2 nos muestra las operaciones que pueden hacerse:
 - **osMessageQueueDelete** : Delete a Message Queue object.
 - **osMessageQueueGet** : Get a Message from a Queue or timeout if Queue is empty.
 - **osMessageQueueGetCapacity** : Get maximum number of messages in a Message Queue.
 - **osMessageQueueGetCount** : Get number of queued messages in a Message Queue.
 - **osMessageQueueGetMsgSize** : Get maximum message size in a Message Queue.
 - **osMessageQueueGetName** : Get name of a Message Queue object.
 - **osMessageQueueGetSpace** : Get number of available slots for messages in a Message Queue.
 - **osMessageQueueNew** : Create and Initialize a Message Queue object.
 - **osMessageQueuePut** : Put a Message into a Queue or timeout if Queue is full.
 - **osMessageQueueReset** : Reset a Message Queue to initial empty state.

Lo usamos en el ejemplo del muestreo

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {

    int16_t val;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    val = HAL_ADC_GetValue(&hadc1);
    osStatus_t res = osMessageQueuePut(queueDatosHandle, &val, 0, 0);
    if(res != osOK){
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    }
    count++;
    if(count== NS){
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
        count = 0;
        xHigherPriorityTaskWoken = pdTRUE;
        osSemaphoreRelease(semSync1Handle);
    }
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Procesamiento de media y std

```
void entryTaskAlta(void *argument){ ...
for (;;) {
    osSemaphoreAcquire (semSynclHandle, osWaitForever);    // Sincronización
    acc=0;          // Reset de variables
    istd = 0;       //
    for(i=0; i<N; i++){    // Cálculo de promedio y desviación estándar
        res = osMessageQueueGet (queueDatosHandle, &val, 0, 0); // Recepción de queue
        if(res == osOK ) acc += ( float)val;                    // Si no hubo errores, sumar
        else HAL_GPIO_WritePin (GPIOC, GPIO_PIN_13, GPIO_PIN_SET); // Si los hubo, prender luz
        dif = ( int)val-prom;    // xi- prom
        istd += dif*dif;        // sum [ (xi-prom)^2 ]
    }
    acc = acc/N;
    prom = acc;                // Actualizacion var global con promedio
    istd = istd/N;
    std = sqrt(istd);          // Actualización var global con desv est
}}
```


Completamos el ejemplo



Procesamiento de interrupciones diferido

- En este ejemplo se repartió la carga de forma “secuencial” para demostrar el uso de semáforos y colas
- Si hay otras tareas para realizar en el OS, se pueden configurar para ejecutar concurrentemente
- Tenemos un factor de utilización del 80%



Notificaciones

Task Notifications

- Hasta ahora los elementos de sincronización son **externos** a las tareas
 - Queue
 - Mutex
 - Semáforo
- Las notificaciones a tareas consisten en un campo opcional en el TBC de las tareas y permiten escribir 1 flag y 1 valor usando una API de FreeRTOS
- A diferencia de los otros elementos, son propias de cada tarea, esto dicta la lógica de funcionamiento (por ejemplo, no se puede notificar **a una** ISR, pero si **desde una** ISR)

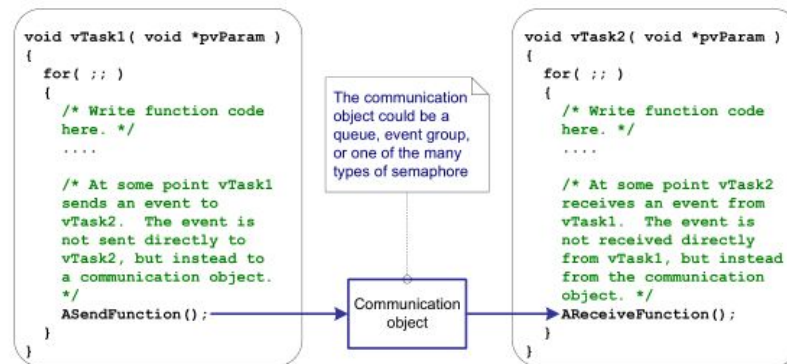
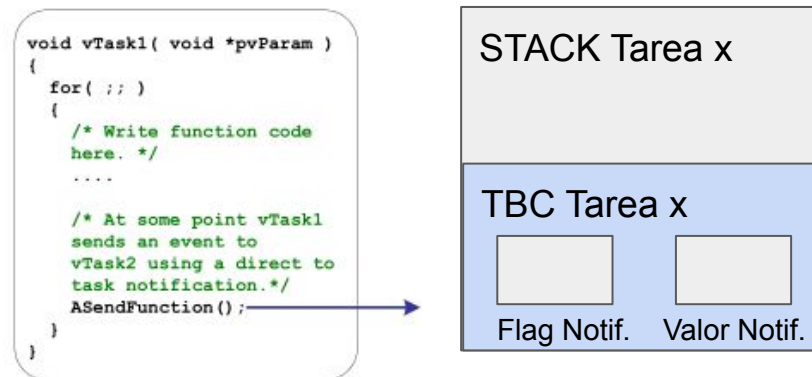
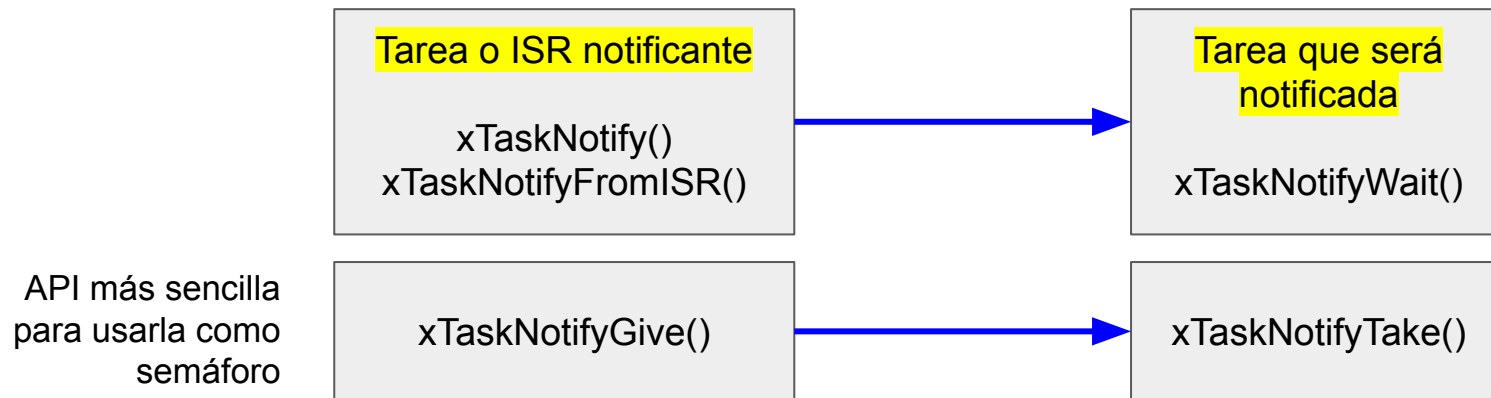


Figure 76 A communication object being used to send an event from one task to another



Task Notifications

When `configUSE_TASK_NOTIFICATIONS` is set to 1, each task has a 'Notification State', which can be either 'Pending' or 'Not-Pending', and a 'Notification Value', which is a 32-bit unsigned integer. When a task receives a notification, its notification state is set to pending. When a task reads its notification value, its notification state is set to not-pending.



Task Notifications

- La API simplificada usa los campos de notificación para emular un semáforo:
 - Setea el campo de notificación
 - Incrementa (en el caso de Give) en uno la variable

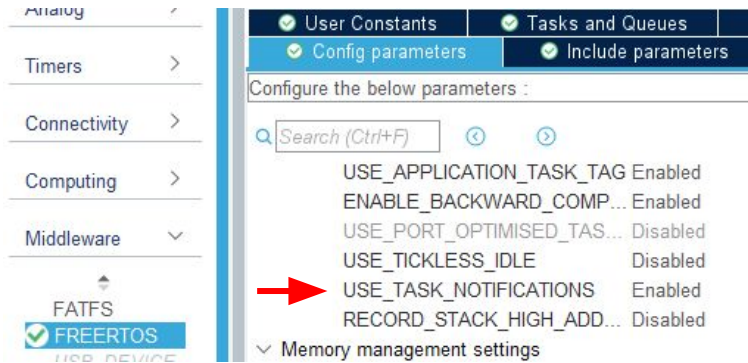
```
BaseType_t xTaskNotifyGive( TaskHandle_t xTaskToNotify );
```

Listing 145. The xTaskNotifyGive() API function prototype

```
void vTaskNotifyGiveFromISR( TaskHandle_t xTaskToNotify,  
                             BaseType_t *pxHigherPriorityTaskWoken );
```

Listing 146. The vTaskNotifyGiveFromISR() API function prototype

Ejemplo de uso.

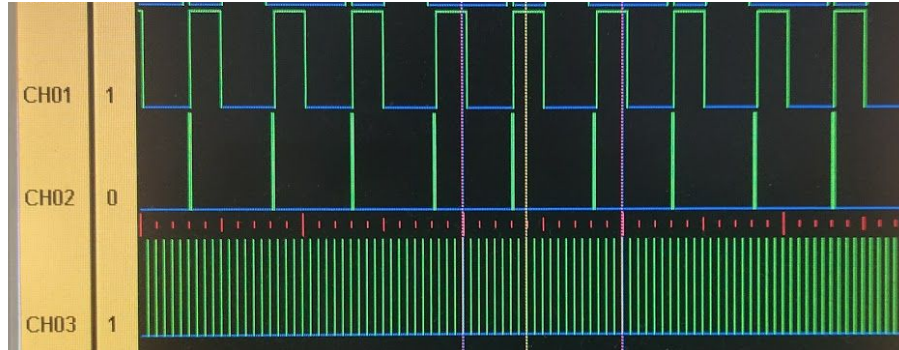


```
for (;;) {
    // uint32_t ulTaskNotifyTake(
    // BaseType_t xClearCountOnExit,
    // TickType_t xTicksToWait);
    ulTaskNotifyTake(pdFALSE, portMAX_DELAY);
    acc=0;
    for(i=0; i<N; i++){ ... }
    acc = acc/N;
    prom = acc;
}
```

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    val = HAL_ADC_GetValue(&hadc1);
    osStatus_t res = osMessageQueuePut(queueDatosHandle, &val, 0, 0);
    if(res != osOK) HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    count++;
    if(count== NS){
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
        count = 0;
        vTaskNotifyGiveFromISR(TaskAltaHandle, &xHigherPriorityTaskWoken);
        xHigherPriorityTaskWoken = pdTRUE;
    }
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

(Reemplazamos el elemento de sincronización del ejemplo anterior)

Ejemplo de uso



`vTaskNotifyGiveFromISR` toma por referencia el valor `xHigherPriorityTaskWoken`. Si es `pdTRUE`, significa que la tarea notificada pasará al estado `RUNNING`

```
// uint32_t ulTaskNotifyTake(  
// BaseType_t xClearCountOnExit,  
// TickType_t xTicksToWait);  
ulTaskNotifyTake(pdFALSE, portMAX_DELAY);  
  
vTaskNotifyGiveFromISR(TaskAltaHandle,  
                        &xHigherPriorityTaskWoken);
```

`ulTaskNotifyTake` decrementa en 1 el valor de notificación, o lo hace 0 si el 1er parámetro es `pdTRUE`. Devuelve como `uint32_t` el valor previo a su borrado o decremento, si se quiere usar como semáforo con conteo.

xTaskNotifyFromISR()

Da un acceso “completo” a la funcionalidad de las notificaciones. Se puede usar para

- Incrementar el valor de notificación (usándola igual que la versión Give)
- Setear un valor particular para señalizar mensajes. Esta funcionalidad está pensada para usarla seteando bits del valor de notificación.
- Sobreescribir el valor de notificación sólo si la tarea receptora lo ha leído desde su última actualización (se comporta como una cola de 1 sólo elemento).
- Sobreescribir el valor de notificación sin importar si se ha leído o no

```

BaseType_t xTaskNotify( TaskHandle_t xTaskToNotify,
                        uint32_t ulValue,
                        eNotifyAction eAction );

BaseType_t xTaskNotifyFromISR( TaskHandle_t xTaskToNotify,
                              uint32_t ulValue,
                              eNotifyAction eAction,
                              BaseType_t *pxHigherPriorityTaskWoken );

```

eNoAction	No hace nada. Equivalente a semáforo binario
eSetBits	Se hace un OR binario entre el valor de notificación que se pasa (ulValue) y el actual
eIncrement	Se incrementa en 1 el valor de notificación actual (ulValue no se usa)
eSetValueWithoutOverwrite	Si la notificación estaba en estado pendiente no se sobreescribe y devuelve pdFAIL
eSetValueWithOverwrite	Se sobreescribe el valor con ulValue sin importar el estado de notificación

xTaskNotifyWait()

Espera con un timeout opcional a que se le envíe una notificación. Sus argumentos permiten:

- **ulBitsToClearOnEntry:** Borrar los bits del valor de notificación al entrar a la función (si no tenía una notificación pendiente). Es decir que para borrar a 0 hay que pasarle 0xFFFFFFFF
- **ulBitsToClearOnExit:** Borrar los bits del valor de notificación al salir de la función, pero previamente se almacena el valor en la variable apuntada por el 3er argumento
- **pulNotificationValue:** Recupera el valor de notificación antes de ser borrado OnExit
- **xTicksToWait:** Timeout en ticks. Puede usarse `pdMS_TO_TICKS()` para introducir el valor en ms o `portMAX_DELAY` para que la espera sea “infinita”

Y su valor de retorno puede ser

- **pdTRUE:** Se recibió la notificación esperada ya sea luego de haber estado esperando en estado bloqueado o previo al llamado de la función
- **pdFALSE:** La función retornó sin haber recibido la notificación (ocurrió el timeout)

```
BaseType_t xTaskNotifyWait( uint32_t ulBitsToClearOnEntry,  
                             uint32_t ulBitsToClearOnExit,  
                             uint32_t *pulNotificationValue,  
                             TickType_t xTicksToWait );
```

Ejemplo modo “semáforo binario”

```
void entryTaskNormal(void *argument)
{
    /* USER CODE BEGIN entryTaskNormal */
    vTaskSetApplicationTaskTag( NULL, (void*) TAG_TASK_NORMAL);
    uint8_t data;
    /* Infinite loop */
    for (;;) {
        HAL_UART_Receive_IT(&huart1, &data, 1);
        xTaskNotifyWait(0U, 0U, NULL, portMAX_DELAY);
        HAL_UART_Transmit(&huart1, "ACK\n\r", 6, 100);
    }
    /* USER CODE END entryTaskNormal */
}
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
    BaseType_t pxHigherPriorityTaskWoken = pdFALSE;
    xTaskNotifyFromISR(TaskNormalHandle, 0U, eNoAction, &pxHigherPriorityTaskWoken);
}
```

Ejemplo con pasaje de datos

```
void entryTaskNormal(void *argument)
{
    ...
    for (;;) {
        if(recibido==1){
            HAL_UART_Receive_IT(&huart1, &data, 1);
            recibido = 0;
        }
        xTaskNotifyWait(0U, 0U, &val_notif, portMAX_DELAY);
        if(val_notif == TX_ACK){
            recibido = 1;
            HAL_UART_Transmit(&huart1, "ACK\n\r", 6, 100);
        }
        else if(val_notif == TX_VAL){
            numchar = sprintf(num,"%d\n\r",prom_int);
            HAL_UART_Transmit(&huart1, num, numchar+1, 100);
        }
    }
    /* USER CODE END entryTaskNormal */
}
```

```
#define TX_ACK 1U
#define TX_VAL 2U
```

```
void entryTaskAlta(void *argument)
{
    ...
    for (;;) {
        ulTaskNotifyTake(pdFALSE, portMAX_DELAY);
        acc=0;
        for(i=0; i<N; i++){...}
        acc = acc/N;
        prom_int = (int)acc;
        xTaskNotify(TaskNormalHandle, TX_VAL,
                    eSetValueWithOverwrite);
    }
}
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
    BaseType_t pxHigherPriorityTaskWoken = pdFALSE;
    xTaskNotifyFromISR(TaskNormalHandle, TX_ACK,
                        eSetValueWithOverwrite,&pxHigherPriorityTaskWoken);
}
```

Tera Term -

File Edit Setu

2018
2018
2017
2018
2017
2018
2017
2017
2018
2018
2017
2017
2017
ACK
2017
2017
2017
2017
2017